

24CSA212 – Full Stack Frameworks

AUTHENTICATION AND AUTHORIZATION,

Authentication strategies available in Express

In the context of web development, authentication and authorization in express are important concepts that play a crucial role in securing applications and controlling access to resources.

Authentication:

Authentication is the process of verifying the identity of a user or entity. It ensures that the user claiming to be a particular person or entity is indeed who they say they are. In web applications, authentication is typically performed when a user tries to access protected resources or perform privileged actions.

There are several commonly used authentication mechanisms:

Username and password: This is the most common method where users provide a username (or email) and a password to prove their identity.

Token-based authentication: Tokens, such as JSON Web Tokens (JWT), are generated and issued to users upon successful authentication. These tokens are then included in subsequent requests to authenticate and authorize the user.

Single sign-on (SSO): SSO allows users to authenticate once and gain access to multiple applications or services without providing their credentials repeatedly.

Authentication strategies available in Express

The authentication process usually involves the following steps:

1. The user provides their credentials.
2. The server verifies the provided credentials.
3. If the credentials are valid, the server generates an authentication token or session for the user.
4. The token/session is sent back to the client and stored for subsequent requests.
5. The client includes the token/session in each request to prove its authenticity.

Authentication strategies available in Express

- Authentication is an important aspect of web development, which ensures that users accessing an application are who they claim to be.
 - In Express, several authentication strategies are available that help you secure your applications by verifying user identities.
-

Different Types of Authentication Patterns:

- Authentication patterns are used to manage user authentication based on whether or not the server keeps track of or maintains user state or data.

There are two types of authentication patterns:

- Stateless Authentication
- Stateful Authentication

Authentication strategies available in Express

1. Stateless Authentication

The server does not store any data or state of the user between requests. It means each request from the client/ User to the server **contains all the data needed** to authenticate the user.

Some Stateless authentication strategies in ExpressJS are

- Basic Authentication
- Token-Based Authentication
- OAuth Authentication (when implemented with stateless tokens)

A. Basic Authentication

Basic Authentication is one of the simplest and most widely used auth strategy across the web.

In express, it involves sending the user's credentials i.e username, and password with each HTTP request coded in "**base64-encoded format**"

Though it is easy to implement, its base64-encoded format can be easily decoded so it is recommended to use this method only when coupled with a secure transport layer such as HTTPS.

Authentication strategies available in Express

You can use express-basic-auth middleware in Express to implement this authentication method.

```
//app.js
const express = require("express");
const basicAuth = require("express-basic-auth");
const app = express();
app.use(
  basicAuth({
    users: { username: "password" },
    challenge: true,
    unauthorizedResponse:
      "Unauthorized access. Please provide valid credentials.",
  })
);
app.get("/secure-data", (req, res) => {
  res.send("This is secure data that requires valid credentials.");
});
const port = 3000;
app.listen(port, () => {
  console.log(`Server is running on http://localhost:${port}`);
});
```

Authentication strategies available in Express

B. Token-Based Authentication:

- It is a more secure and scalable alternative to basic authentication.
- JSON Web Tokens(JWT) are commonly used in Express to implement token-based authentication.
- When the user logs in, the server generates a token containing the user's information.
- Then the server sends the token to the client in response.
- The client stores the token in the form of a cookie or local storage.
- In the subsequent request, the client includes this token in the header, enabling the server to validate the user.
- The features of token-based auth include expiration time and digital signatures enhancing the security and integrity of the data.

```
const jwt = require("jsonwebtoken");
const token = jwt.sign({ userId: "246" }, "secretKey", { expiresIn: "2h" }); // Generating a token

jwt.verify(token, "secretKey", (err, decoded) => { // Verifying the token[Authorization]
  if (err) {
    // Token is invalid
  } else {
    // Token is valid, and decoded contains user information
  }
});
```

Authentication strategies available in Express

C. OAuth Authentication

- OAuth (Open Authorization) is an industry-standard protocol for authentication.
- OAuth enables users to grant third-party applications limited access to their resources without sharing credentials (passwords).
- In Express JS with the help of Passport.JS, one can integrate OAuth authentication strategies for popular providers such as Google, Twitter, or Facebook.
- OAuth leverages the existing credentials from trusted providers, offering a secure user experience.

Authentication strategies available in Express

2. Stateful Authentication

- In this authentication pattern, the Server stores the state or data of the user about the user session or authentication state.
- The server uses this information/ Data to authenticate the user.
- Stateful authentication uses cookies to identify the user with their request.
- In Express.js Authentication strategies such as Passport.js and Middleware-based authentication can be both stateful or stateless depending on the use case and implementation chosen by developers.

A. Passport.js Middleware

- Passport.js is the authentication middleware for Node.js applications, especially for frameworks like ExpressJS.
- It supports various strategies such as local authentication, OAuth, OpenID, and others.
- It's flexible to allow developers to choose the strategies that align with their web app the best.
- Passport.JS delegates the intricacies of different strategies to specialized modules.
- This modular design makes it easy to integrate for changing requirements.

Authentication strategies available in Express

```
const passport = require('passport');
```

```
const LocalStrategy = require('passport-local').Strategy;
```

```
passport.use(new LocalStrategy(
  (username, password, done) => {
    // Validate user credentials
    // If valid, call done(null, user)
    // Otherwise, call done(null, false, { message: 'Incorrect credentials.' })
  }
));
```

```
app.post('/login', passport.authenticate('local', {
  successRedirect: '/dashboard',
  failureRedirect: '/login',
  failureFlash: true
}));
```

Authentication strategies available in Express

B. Middleware-Based Authentication

- Middleware-based authentications involve using custom middleware functions for authorization
- Middleware functions are the functions that have access to the request, response, and the next function in the application's request-response cycle
- They can modify request and response objects, call the next function, and end the request-response cycle in the stack.
- Middleware-based authentication offers maximum flexibility among others. It allows developers to customize authentication logic to specific application requirements.

Conclusion

- Express provides many strategies for authentication and authorization, each works at specific use cases and requirements.
- The choice of choosing the authentication strategy for your application depends on the nature of your app, industry standards, and the sensitivity of user data.

Authentication strategies available in Express

Purpose of using Authentication:

- **User Identification:** Authentication allows the identification of individual users accessing the web application. It verifies the user's identity by validating their credentials, such as a username and password combination or a token.
- **Access Control:** Authentication helps in enforcing access control by determining what resources or routes a user is allowed to access based on their authenticated status and user roles or permissions. It prevents unauthorized users from accessing sensitive information or performing restricted actions.
- **User Sessions:** Authentication mechanisms in Express.js typically involve the creation and management of user sessions. When a user logs in successfully, a session is established, and a unique session identifier (usually stored as a cookie) is associated with that user. Subsequent requests from the same user include the session identifier, allowing the server to maintain the session state and provide personalized experiences.
- **Protecting Routes:** With authentication, you can secure specific routes or endpoints within your Express.js application. Certain routes may require authentication to be accessed, while others can be publicly available. By enforcing authentication on protected routes, you ensure that only authenticated users can access or modify sensitive data or perform critical operations.
- **User Management:** Authentication is also crucial for user management functionalities, such as registration, password resets, and user profile updates. By verifying the user's identity, you can securely handle these operations and ensure that only the authenticated user can modify their information.
- **Security:** Authentication is a vital security measure to protect against unauthorized access, data breaches, and potential attacks like identity theft, session hijacking, or cross-site scripting (XSS). By

Authorization in Express

Authorization:

Authorization determines the actions or resources a user is allowed to access within an application after successful authentication.

Once a user's identity is established, the authorization process enforces access control rules and permissions to ensure that the user can only perform their authorized actions.

Authorization is typically based on roles, permissions, or policies assigned to users.

Some common authorization mechanisms include:

Role-based access control (RBAC): Users are assigned specific roles (e.g., admin, moderator, user), and each role has a set of predefined permissions that determine what actions or resources they can access.

Attribute-based access control (ABAC): Access control decisions are made based on various attributes associated with users, resources, and environmental conditions.

Policy-based access control: Access control policies are defined in a centralized manner, specifying the conditions and rules for granting or denying access to

Authorization in Express

The authorization process generally involves the following steps:

1. The server receives a request from an authenticated user.
2. The server identifies the user and retrieves their assigned roles or permissions.
3. The server evaluates the requested action or resource against the user's roles or permissions.
4. If the user is authorized, the server allows the action or provides access to the requested resource; otherwise, it denies access and returns an appropriate response.

Authorization in Express

Benefits of authorization:

- **Security:** Authorization helps protect sensitive data and resources by allowing only authorized users to access them. By defining and enforcing access control policies, organizations can ensure that confidential information remains secure and is only accessible to those with proper permissions.
- **Privacy:** Authorization allows organizations to control access to personal or private information. By specifying who can view, modify, or delete certain data, organizations can safeguard the privacy of individuals and comply with data protection regulations.
- **Data Integrity:** Authorization helps maintain the integrity of data by ensuring that only authorized users can modify or delete information. By enforcing proper access controls, organizations can prevent unauthorized or malicious changes to critical data, preserving its accuracy and reliability.
- **Compliance:** Authorization plays a crucial role in meeting regulatory and compliance requirements. Many industries have specific regulations that mandate access control and the protection of sensitive data. By implementing proper authorization mechanisms, organizations can demonstrate compliance and mitigate the risk of legal or regulatory penalties.
- **Least Privilege Principle:** Authorization follows the principle of least privilege, which states that users should only have the minimum privileges necessary to perform their tasks. By granting specific permissions based on roles or attributes, organizations can limit potential damage caused by insider threats, accidental misuse, or unauthorized access.
- **User Management:** Authorization facilitates user management by allowing administrators to define and manage user roles, permissions, and access levels. This simplifies the process of granting or revoking access to resources as user roles or responsibilities change over time.
- **Auditing and Accountability:** Authorization mechanisms often provide logging and auditing capabilities, allowing organizations to track and monitor user actions. This helps in identifying any unauthorized access attempts, detecting suspicious activities, and providing an audit trail for investigations or compliance purposes.
- Overall, the purpose of the authorization is to control and manage access to resources, ensuring that only authorized users can perform specific actions or access sensitive information. By implementing robust authorization mechanisms,

Authentication and Authorization in Express - Conclusion

Here's a summary of the key points regarding authentication and authorization in Express.js:

Authentication:

- Authentication is the process of verifying the identity of a user or entity accessing a system or resource.
- In Express.js, authentication involves validating user credentials, establishing user sessions, or verifying tokens.
- Authentication ensures that only authorized users can access protected resources or perform specific actions.
- It helps in user identification, access control, user session management, and securing sensitive routes and resources.
- Common authentication mechanisms in Express.js include username/password, JSON Web Tokens (JWT), and OAuth.

Authorization:

- Authorization is the process of granting or denying access to specific resources or functionalities based on user roles or permissions.
- It comes after authentication to determine what an authenticated user can or cannot do within the application.
- Authorization is typically implemented by associating roles or permissions with users and validating them during the authentication process or within route handlers.